CAMPUS RIDE

Design Document

Project: Real-Time Campus Mobility and Ride Management Platform

Technology Stack: MERN (MongoDB, Express.js, React.js, Node.js)

---

1. Problem Understanding

Background and Objectives

Large educational campuses such as IIT Roorkee require an efficient transportation system for students, faculty, staff, and visitors. Traditional campus transportation relies heavily on manual coordination between passengers and drivers, leading to:

- Difficulty locating available drivers
- Delayed ride allocation
- Lack of ride visibility and tracking
- No centralized scheduling mechanism
- Manual payment handling
- Absence of performance monitoring and feedback

To address these challenges, **Campus Ride** provides a centralized web-based platform connecting passengers and drivers in real time.

Objectives

- Instant ride booking within campus
- Real-time driver-passenger matching
- Advance ride scheduling
- Live ride status updates and tracking
- Ride and payment history management
- Driver ratings and feedback
- Driver performance analytics

Scope

Passenger Services

- Registration and login
- Ride booking, cancellation, and scheduling
- Ride tracking and history
- Payment history
- Driver ratings and feedback

Driver Services

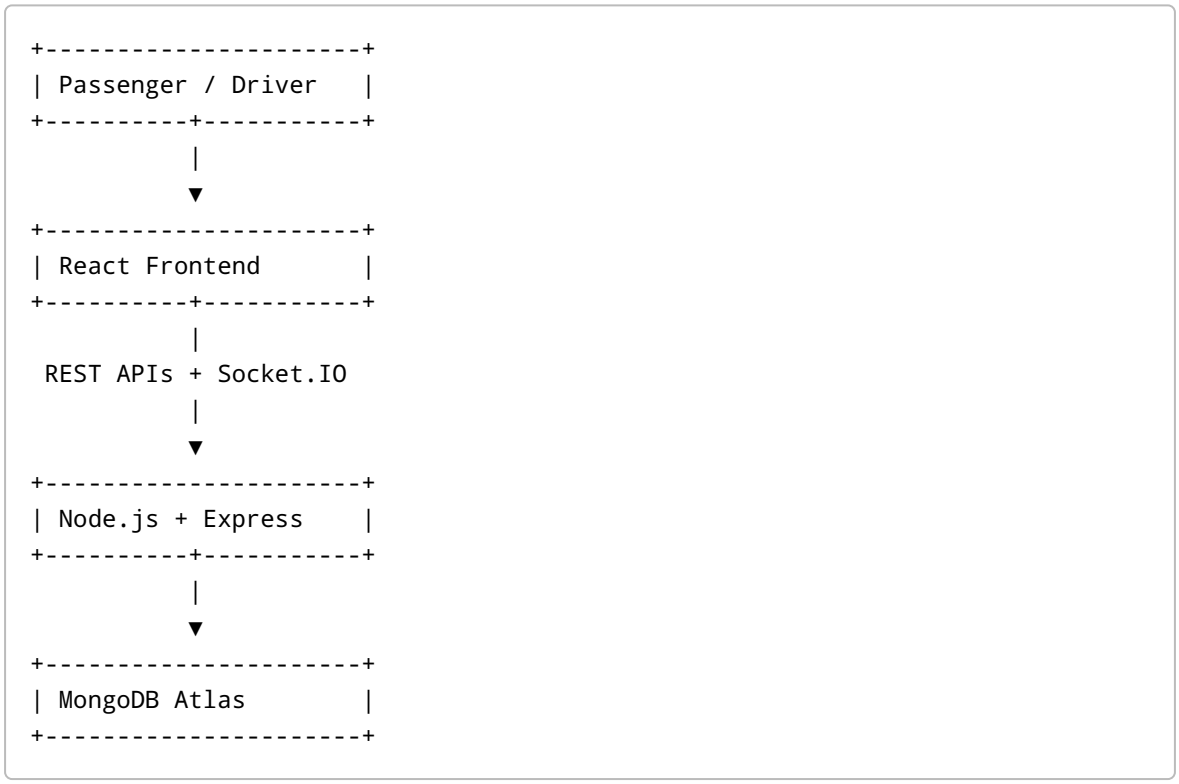
- Registration and login
- Online/offline availability management
- Ride acceptance and status updates
- Live location updates
- Earnings tracking and UPI management
- Performance analytics

---

2. System Architecture

Architectural Overview

The system follows a MERN-based client-server architecture.



Frontend

Technologies: React.js, React Router DOM, Axios, Socket.IO Client, React Hot Toast, Leaflet Maps

Responsibilities:

- Authentication
- Passenger and driver dashboards
- Ride management
- Payment history
- Live maps and notifications

Backend

Technologies: Node.js, Express.js, Socket.IO, JWT, Bcrypt, Mongoose

Responsibilities:

- Authentication and authorization
- Ride lifecycle management
- Driver and rating management
- Analytics generation
- Real-time communication

Real-Time Communication

Socket.IO enables instant synchronization between users through events such as:

- User registration
- Ride requests
- Ride acceptance
- Ride status updates
- Driver location updates
- Driver availability updates
- Ride cancellation

---

3. Database Schema

MongoDB Atlas serves as the primary database.

User Collection

Stores passenger and driver information.

Field	Type
_id	ObjectId
name	String
email	String
password	String
role	String
phone	String
vehicleNumber	String
vehicleType	String
licenseNumber	String

Field	Type
upiId	String
isOnline	Boolean
isBusy	Boolean
averageRating	Number
totalRatings	Number
totalRides	Number
currentLocation	Object
createdAt	Date
updatedAt	Date

Ride Collection

Stores ride lifecycle information.

Field	Type
passenger	ObjectId
driver	ObjectId
pickupLocation	Object
destination	Object
status	String
fare	Number
distance	Number
scheduledTime	Date
isScheduled	Boolean
startTime	Date
endTime	Date
paymentMethod	String
paymentStatus	String
cancelledBy	String
cancelReason	String
createdAt	Date
updatedAt	Date

Rating Collection

Stores passenger feedback.

Field	Type
ride	ObjectId
passenger	ObjectId
driver	ObjectId
rating	Number
feedback	String
createdAt	Date
updatedAt	Date

Scheduling Design

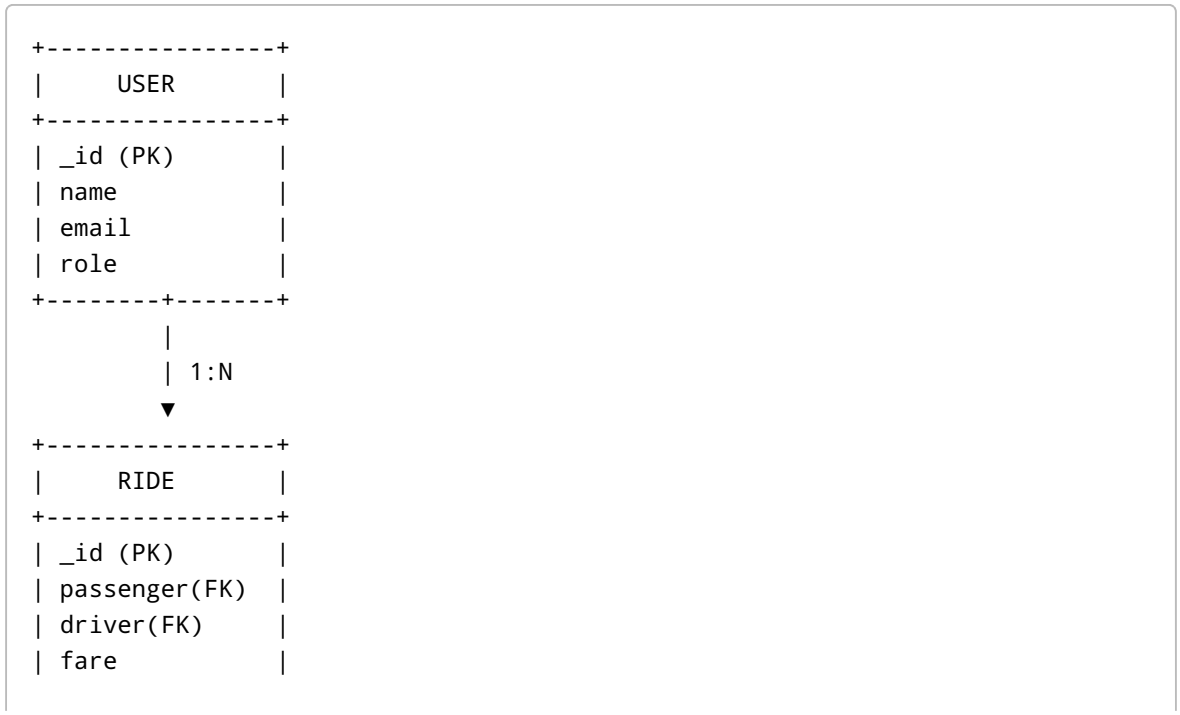
Although a separate Schedule schema was initially considered, scheduled rides are managed directly within the Ride collection using:

- isScheduled
- scheduledTime

This reduces redundancy and simplifies ride management.

---

4. Entity Relationship Diagram (ERD)



```
| status         |
+--------+-------+
         |
         | 1:1
         ▼
+----------------+
|    RATING      |
+----------------+
| _id (PK)       |
| ride (FK)      |
| passenger(FK)  |
| driver(FK)     |
| rating         |
+----------------+
```

Relationships

- One passenger can create multiple rides.
- One driver can complete multiple rides.
- One completed ride can receive one rating.
- One driver can receive multiple ratings.

---

5. API Overview

Authentication APIs

Method	Endpoint	Purpose
POST	/api/auth/register	Register user
POST	/api/auth/login	Login
GET	/api/auth/profile	Get profile
PUT	/api/auth/profile	Update profile

Ride APIs

Method	Endpoint	Purpose
POST	/api/rides	Request ride
GET	/api/rides/available	Available rides
GET	/api/rides/my	Ride history
GET	/api/rides/active	Active ride
PUT	/api/rides//accept	Accept ride
PUT	/api/rides//status	Update status

Method	Endpoint	Purpose
PUT	/api/rides//cancel	Cancel ride
GET	/api/rides/analytics	Analytics

Driver APIs

Method	Endpoint	Purpose
PUT	/api/drivers/toggle-online	Toggle availability
PUT	/api/drivers/location	Update location
GET	/api/drivers/online	Available drivers
GET	/api/drivers/stats	Driver statistics
GET	/api/drivers/stats/	Public statistics
PUT	/api/drivers/upi	Update UPI
GET	/api/drivers//public	Driver information

Rating APIs

Method	Endpoint	Purpose
POST	/api/ratings	Submit rating
GET	/api/ratings/driver/	Driver ratings

---

6. Design Decisions

MERN Stack

Chosen for:

- Single-language development using JavaScript
- Faster development cycle
- Strong ecosystem and scalability

MongoDB

Selected because of:

- Flexible schema design
- Efficient handling of location data
- Easy scalability and Mongoose integration

JWT Authentication

Provides:

- Stateless authentication
- Reduced server-side overhead
- Better scalability

Password Security

Passwords are encrypted using bcrypt to prevent plain-text storage and improve security.

Socket.IO

Used for:

- Instant ride notifications
- Ride acceptance updates
- Live ride tracking
- Driver location updates

Ride-Centric Scheduling

Scheduled rides are stored within the Ride collection using `isScheduled` and `scheduledTime`, reducing complexity and duplicate records.

Role-Based Access Control

Separate permissions for passengers and drivers improve security and simplify application management.

Distance-Based Fare Model

Distance	Fare
$\leq$ 0.5 km	₹10
$\leq$ 1.0 km	₹20
$\leq$ 1.5 km	₹30
> 1.5 km	₹40

Benefits:

- Transparent pricing
- Faster fare computation
- Simple user experience

Campus Ride provides a centralized, secure, and scalable solution for campus transportation. Through the MERN stack, Socket.IO-based real-time communication, JWT authentication, and an optimized

database design, the platform enables efficient ride management while delivering a seamless experience for both passengers and drivers.